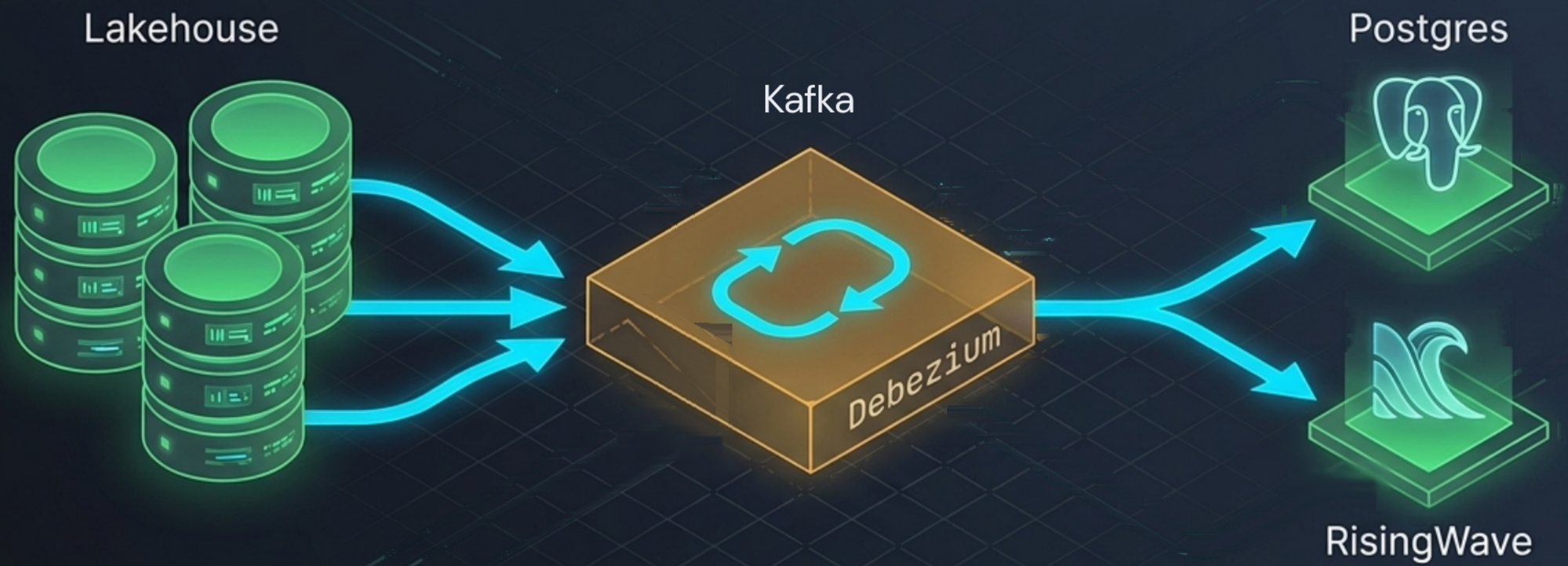
Architecting a Zero-DBU, CDC-Native Reverse ETL Pipeline

How we bypassed Databricks compute to stream Change Data Feed (CDF) events to downstream operational systems at scale.

TARGET_AUDIENCE: [Data Eng, Data Arch, Tech Lead] // CLEARANCE: Open



TL;DR Execution Summary

Achieved Zero-DBU extraction bypassing Databricks compute via delta-rs direct-to-storage reads.

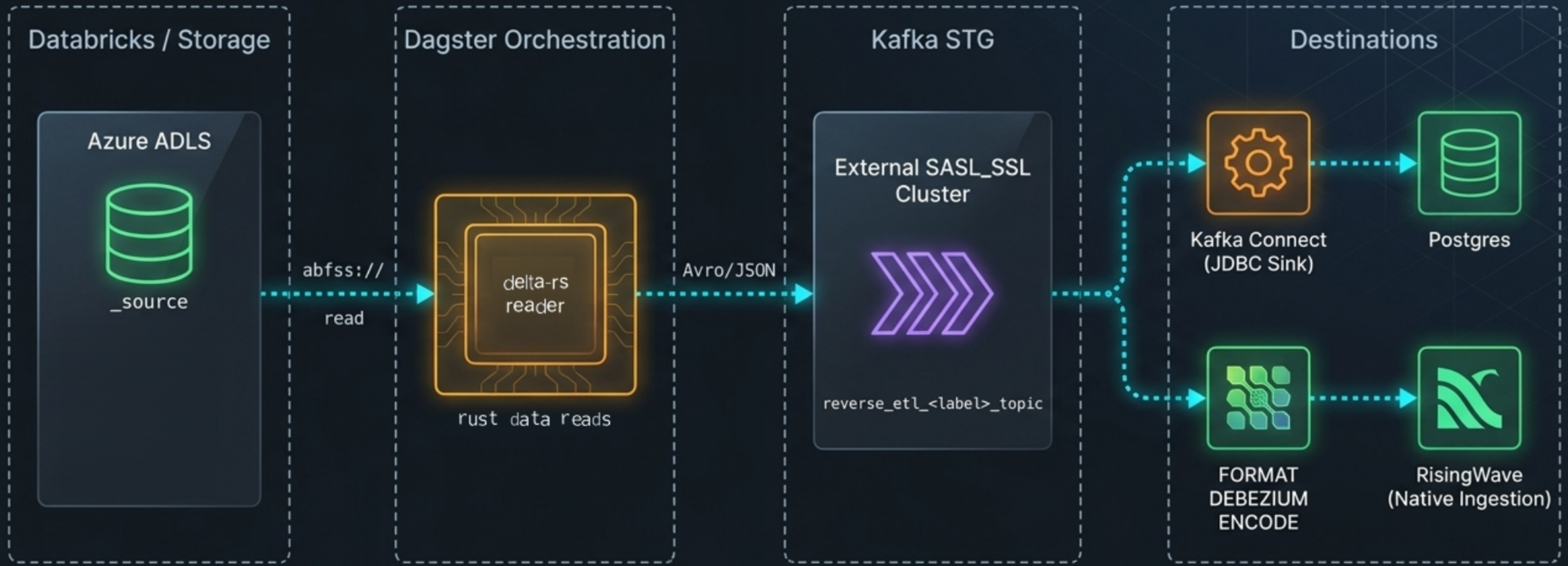
Built native Debezium change events from Delta snapshot diffs, streamed via Kafka.

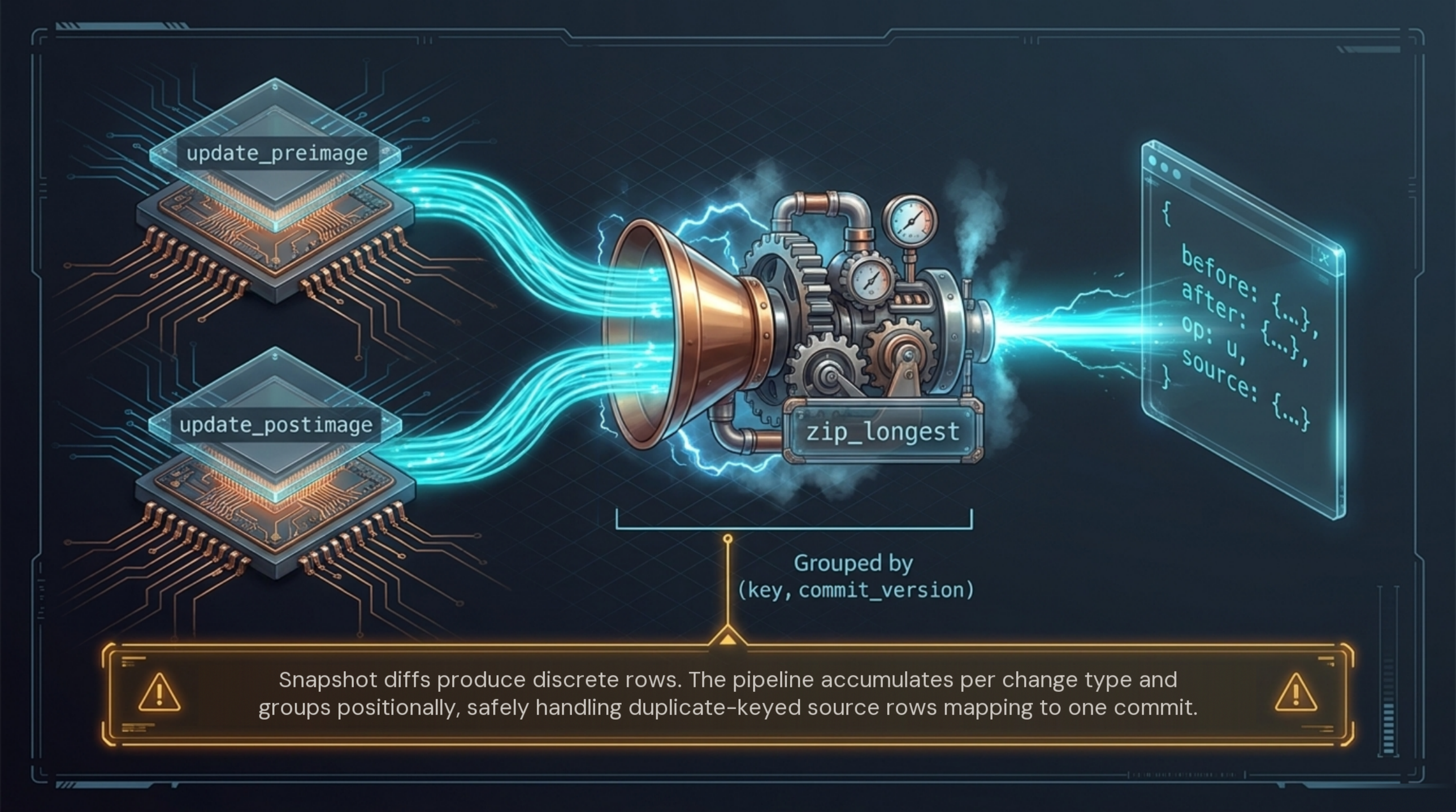
Delivered Multi-target syncs (Postgres + RisingWave) from a single Kafka topic, orchestrated by Dagster.

Evaluation Criteria	drt (v1.0.0)	dlt	Custom Dagster Component
Destination Support	❌ Direct to DB only (No Kafka)	❌ Rejects Kafka destinations	✅ Writes to Kafka
Extraction Method	❌ Mirror mode (High DBU cost)	❌ Lacks CDF source	✅ Diffs Delta snapshots (zero DBU)
Data Semantics	❌ Tracking tables (Loses explicit deletes)	❌ Row-level upserts	✅ Native Debezium CDC envelopes (c/u/d)

Custom Dagster component: no tool covers Delta change extraction, Kafka delivery and zero DBU.

End-to-end architecture from Delta storage to downstream consumers




update_preimage
update_postimage
zip_longest
{
before: {...},
after: {...},
op: u,
source: {...}
}
Grouped by
(key, commit_version)
Snapshot diffs produce discrete rows. The pipeline accumulates per change type and groups positionally, safely handling duplicate-keyed source rows mapping to one commit.

Avro encoding shrinks message payloads by over 90 percent

Requirement:	Constraint:
Shared staging Apicurio registry.	RisingWave requires temporal workaround (timestamp ingested as VARCHAR, generated as TIMESTAMPTZ).

One Topic, Multiple Readers. Each consumer tracks its own offsets independently.

1

Databricks

`ALTER TABLE... ADD COLUMN`

2

Dagster Asset

Reads live Delta schema

3

Dagster Asset

Registers new schema version to Apicurio Registry

4

Dagster Asset

Executes ALTER TABLE on RisingWave

5

Dagster Asset

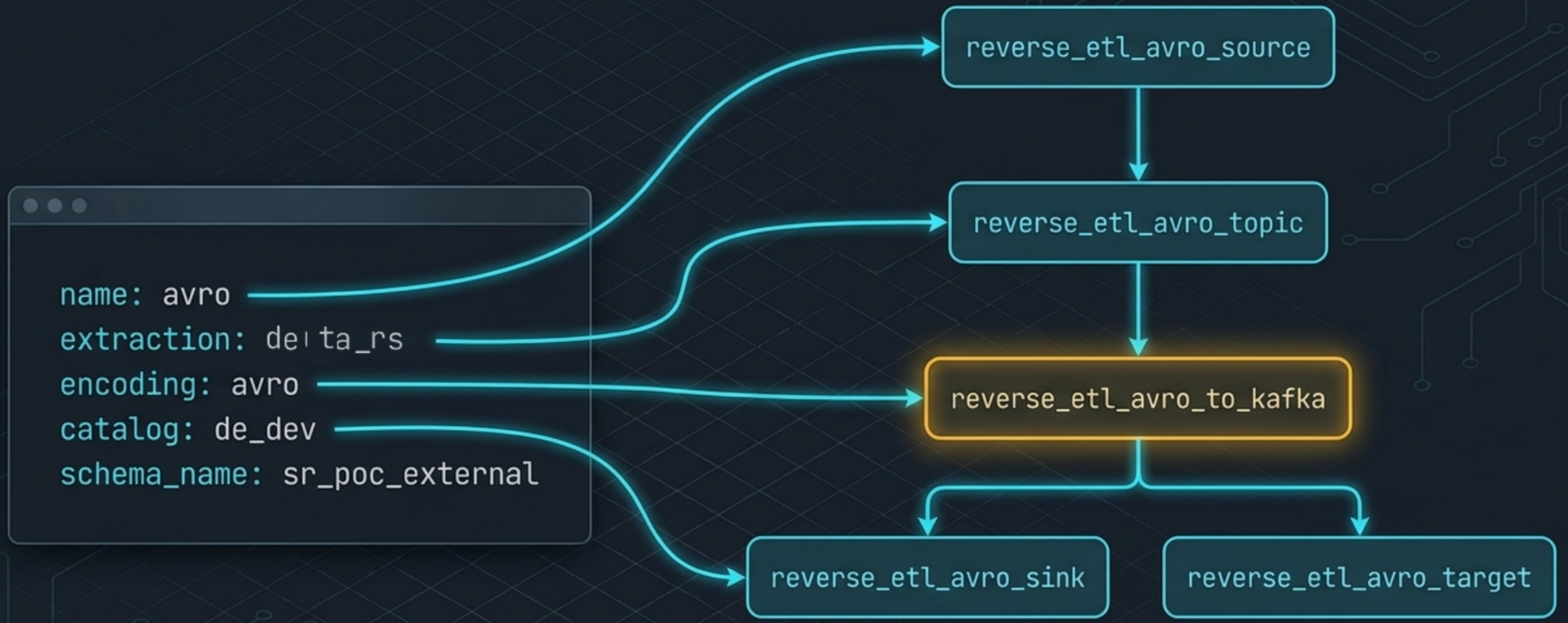
Produces Kafka events

6

Postgres Sink

Auto-adds column via schema.evolution=basic

Crucial Constraint: `RisingWave` must be altered before the first event carrying the new column is produced, otherwise ingested rows default to NULL.



The ReverseEtlSyncConfig object dynamically generates assets. A single label ensures zero name collisions, allowing isolated parallel syncs.

Setup

Idempotent `IF NOT EXISTS` creation of tables and topics.

Sync

Manual trigger from Dagster. The watermark is a Postgres row.. One trigger per change.

Check

The `sink_healthy` scheduled asset check. Prevents silent lag by polling the Connect REST API every 5 minutes.

Reset

Destructive teardown. Safely isolated via strict `reset_name_prefixes` allowlists.

Databricks SQL Path (Baseline)

Databricks SQL Warehouse

delta-rs Path (Zero-DBU)

delta-rs reader

The State Row Decoupling

Source Table
UniForm / Iceberg enabled

Crucial Fix: delta-rs diffs the table's snapshots with zero warehouse compute. The source table can be Delta or UniForm, but the watermark is a Postgres row (`reverse_etl_sync_state`), appended only after producing, so a full sync meters 0.0 Databricks seconds.

State/Watermark Row
Postgres

Surrogate Identity Columns Required.

Delta Row Tracking is not exposed in the CDF. Syncs require an explicitly generated `rid` identity column from table creation to process deterministic upserts and deletes.

Comprehensive Schema Evolution.

The latest version now supports all types of schema evolution, including dropping columns and widening types. Changes are handled seamlessly without breaking downstream RisingWave ingestion or requiring special opt-ins.

Timestamp Zone Formatting.

Collecting `TimestampType` triggers local timezone shifts in delta-rs. Timestamps are formatted purely as `io.debezium.time.ZonedTimestamp` strings with 6 fractional digits to preserve UTC fidelity across systems.